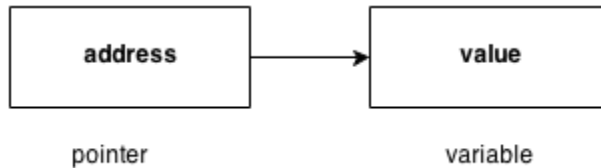


POINTERS:

Introduction

Definition:

Pointer is a variable that stores/hold address of another variable of same data type/ it is also known as locator or indicator that points to an address of a value. A pointer is a derived data type in C



Benefit of using pointers

- Pointers are more efficient in handling Array and Structure.
- Pointer allows references to function and thereby helps in passing of function as arguments to other function.
- It reduces length and the program execution time.
- It allows C to support dynamic memory management.

Declaration of Pointer

```
data_type* pointer_variable_name;
```

```
int* p;
```

Note: void type pointer works with all data types, but isn't used often.

Initialization of Pointer variable

Pointer Initialization is the process of assigning address of a variable to **pointer** variable. Pointer variable contains address of variable of same data type

```
int a = 10 ;
```

```
int *ptr ;    //pointer declaration
```

```
ptr = &a ;    //pointer initialization
```

or,

```
int *ptr = &a ;    //initialization and declaration together
```

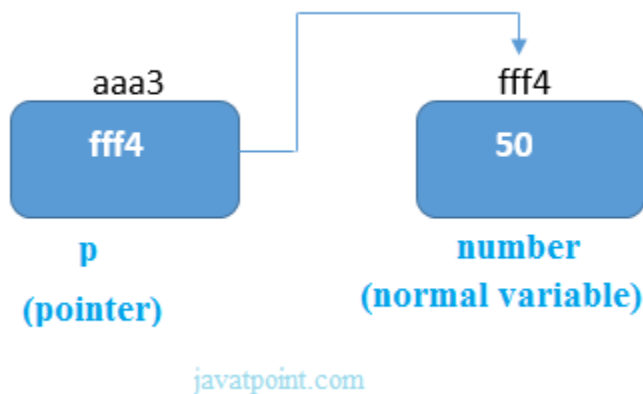
Note: Pointer variable always points to same type of data.

```
float a;
```

```
int *ptr;
```

```
ptr = &a;    //ERROR, type mismatch
```

Above statement defines, p as pointer variable of type int. **Pointer example**



As you can see in the above figure, pointer variable stores the address of number variable i.e. fff4. The value of number variable is 50. But the address of pointer variable p is aaa3.

By the help of * (**indirection operator**), we can print the value of pointer variable p.

Reference operator (&) and Dereference operator (*)

& is called reference operator. It gives you the address of a variable. There is another operator that gets you the value from the address, it is called a dereference operator (*).

Symbols used in pointer

Symbol	Name	Description
& (ampersand sign)	address of operator	determines the address of a variable.
* (asterisk sign)	indirection operator	accesses the value at the address.

Dereferencing of Pointer

Once a pointer has been assigned the address of a variable. To access the value of variable, pointer is dereferenced, using the **indirection operator** *.

```
int a,*p;  
  
a = 10;  
  
p = &a;  
  
printf("%d",*p); //this will print the value of a.  
  
printf("%d",&a); //this will also print the value of a.  
  
printf("%u",&a); //this will print the address of a.  
  
printf("%u",p); //this will also print the address of a.  
  
printf("%u",&p); //this will also print the address of p.
```

KEY POINTS TO REMEMBER ABOUT POINTERS IN C:

- Normal variable stores the value whereas pointer variable stores the address of the variable.
- The content of the C pointer always be a whole number i.e. address.
- Always C pointer is initialized to null, i.e. `int *p = null`.
- The value of null pointer is 0.
- & symbol is used to get the address of the variable.
- * symbol is used to get the value of the variable that the pointer is pointing to.
- If a pointer in C is assigned to NULL, it means it is pointing to nothing.
- Two pointers can be subtracted to know how many elements are available between these two pointers.
- But, Pointer addition, multiplication, division are not allowed.
- The size of any pointer is 2 byte (for 16 bit compiler).

Example:

```
#include <stdio.h>

#include <conio.h>

void main(){

int number=50;

int *p;

clrscr();

p=&number;//stores the address of number variable

printf("Address of number variable is %x \n",&number);

printf("Address of p variable is %x \n",p);

printf("Value of p variable is %d \n",*p);

getch();

}
```

Output

Address of number variable is fff4

Address of p variable is fff4

Value of p variable is 50

Example:

```
#include <stdio.h>

int main()

{

int *ptr, q;

q = 50;
```

```
/* address of q is assigned to ptr */  
ptr = &q;  
/* display q's value using ptr variable */  
printf("%d", *ptr);  
return 0;  
}
```

Output

50

Example:

```
#include <stdio.h>  
  
int main()  
{  
    int var =10;  
    int *p;  
    p= &var;  
    printf ( "\n Address of var is: %u", &var);  
    printf ( "\n Address of var is: %u", p);  
    printf ( "\n Address of pointer p is: %u", &p);  
    /* Note I have used %u for p's value as it should be an address*/  
    printf( "\n Value of pointer p is: %u", p);  
    printf ( "\n Value of var is: %d", var);  
    printf ( "\n Value of var is: %d", *p);  
    printf ( "\n Value of var is: %d", *( &var));  
}
```

Output:

Address of var is: 00XBBA77

Address of var is: 00XBBA77

Address of pointer p is: 77221111

Value of pointer p is: 00XBBA77

Value of var is: 10

Value of var is: 10

Value of var is: 10

NULL Pointer

A pointer that is not assigned any value but NULL is known as NULL pointer. If you don't have any address to be specified in the pointer at the time of declaration, you can assign NULL value.

Or

It is always a good practice to assign a NULL value to a pointer variable in case you do not have an exact address to be assigned. This is done at the time of variable declaration. A pointer that is assigned NULL is called a **null** pointer. `int *p=NULL;`

Note: The NULL pointer is a constant with a value of zero defined in several standard libraries/ in most the libraries, the value of pointer is 0 (zero)

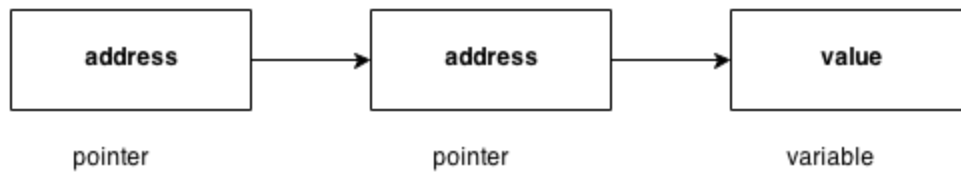
Example:

The value of ptr is 0

Pointers for Inter-Function Communication**Pointers to Pointers**

Pointers can point to other pointers /pointer refers to the address of another pointer.

pointer can point to the address of another pointer which points to the address of a value.

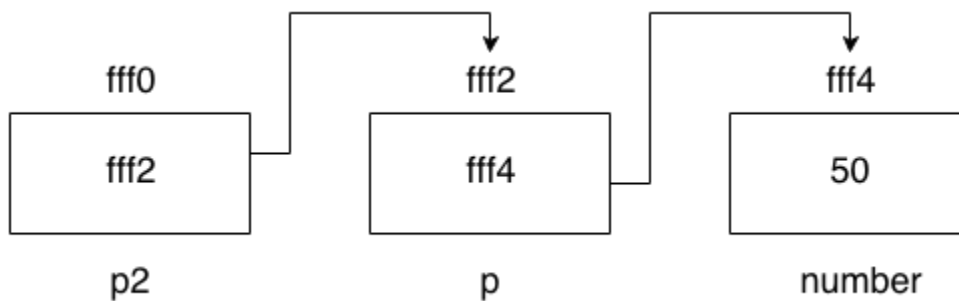


syntax of pointer to pointer

```
int **p2;
```

pointer to pointer example

Let's see an example where one pointer points to the address of another pointer.



Example:

```
#include <stdio.h>

#include <conio.h>

void main(){
    int number=50;

    int *p;//pointer to int
    int **p2;//pointer to pointer

    clrscr();

    p=&number;//stores the address of number variable

    p2=&p;

    printf("Address of number variable is %x \n",&number);

    printf("Address of p variable is %x \n",p);
}
```

```

printf("Value of *p variable is %d \n",*p);
printf("Address of p2 variable is %x \n",p2);
printf("Value of **p2 variable is %d \n",**p);
getch();
}

```

Output

Address of number variable is fff4

Address of p variable is fff4

Value of *p variable is 50

Address of p2 variable is fff2

Value of **p variable is 50

Arrays and Pointers

When an array is declared, compiler allocates sufficient amount of memory to contain all the elements of the array. Base address which gives location of the first element is also allocated by the compiler.

Suppose we declare an array arr,

```
int arr[5]={ 1, 2, 3, 4, 5 };
```

Assuming that the base address of **arr** is 1000 and each integer requires two byte, the five element will be stored as follows

element	arr[0]	arr[1]	arr[2]	arr[3]	arr[4]
Address	1000	1002	1004	1006	1008

Here variable **arr** will give the base address, which is a constant pointer pointing to the element, **arr[0]**. Therefore **arr** is containing the address of **arr[0]** i.e 1000.

arr is equal to **&arr[0]** // by default

We can declare a pointer of type int to point to the array arr.

```
int arr[5]={ 1, 2, 3, 4, 5 };
```

```
int *p;
```

```
p = arr;
```

```
or p = &arr[0]; //both the statements are equivalent.
```

Now we can access every element of array **arr** using **p++** to move from one element to another.

NOTE : You cannot decrement a pointer once incremented. **p--** won't work.

Pointer to Array

we can use a pointer to point to an Array, and then we can use that pointer to access the array. Lets have an example,

```
int i;
```

```
int a[5] = {1, 2, 3, 4, 5};
```

```
int *p = a; // same as int*p = &a[0]
```

```
for (i=0; i<5; i++)
```

```
{
```

```
    printf("%d", *p);
```

```
    p++;
```

```
}
```

In the above program, the pointer ***p** will print all the values stored in the array one by one. We can also use the Base address (**a** in above case) to act as pointer and print all the values.

Replacing the **printf("%d", *p);** statement of above example, with below mentioned statements. Lets see what will be the result.

printf("%d", a[i]); → **prints the array, by incrementing index**

printf("%d", i[a]); → **this will also print elements of array**

printf("%d", a+i); → **This will print address of all the array elements**

printf("%d", *(a+i)); → **Will print value of array element.**

printf("%d", *a); → **will print value of a[0] only**

a++; → **Compile time error, we cannot change base address of the array.**

Relation between Arrays and Pointers

Consider an array:

```
int arr[4];
```

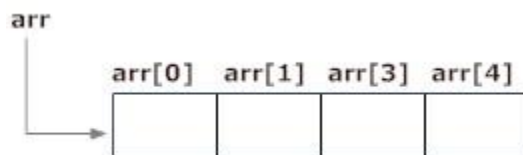


Figure: Array as Pointer

In C programming, name of the array always points to address of the first element of an array.

In the above example, **arr** and **&arr[0]** points to the address of the first element.

&arr[0] is equivalent to arr

Since, the addresses of both are the same, the values of **arr** and **&arr[0]** are also the same.

arr[0] is equivalent to *arr (value of an address of the pointer)

Similarly,

&arr[1] is equivalent to (arr + 1) AND, arr[1] is equivalent to *(arr + 1).

&arr[2] is equivalent to (arr + 2) AND, arr[2] is equivalent to *(arr + 2).

&arr[3] is equivalent to (arr + 3) AND, arr[3] is equivalent to *(arr + 3).

.

.

&arr[i] is equivalent to (arr + i) AND, arr[i] is equivalent to *(arr + i).

Example: Program to find the sum of six numbers with arrays and pointers

```
#include <stdio.h>

int main()
{
    int i, classes[6], sum = 0;
    printf("Enter 6 numbers:\n");
    for(i = 0; i < 6; ++i)
    {
        // (classes + i) is equivalent to &classes[i]
        scanf("%d", (classes + i));
        // *(classes + i) is equivalent to classes[i]
        sum += *(classes + i);
    }
    printf("Sum = %d", sum);
    return 0;
}
```

Output